

DATA ENGINEERING COURSE

JSON & Data Serialization

CLASS NOTES

Instructor: Tanishq Tyagi

Topics Covered

- Part 1: What is JSON? JSON vs Python Dictionary
- Part 2: Serialization & Deserialization
- Part 3: Working with JSON in Python (dump, load, dumps, loads)
- Part 4: Student Data Management
- Part 5: Handling Nested JSON
- Practical: CSV to JSON Conversion

Table of Contents

1. Session Overview & Learning Objectives
2. Part 1: What is JSON?
3. Part 2: Serialization & Deserialization
4. Part 3: Working with JSON in Python
 - 3.1 json.dump() — Write to File
 - 3.2 json.load() — Read from File
 - 3.3 json.dumps() — To JSON String
 - 3.4 json.loads() — From JSON String
5. Part 4: Student Data Management
6. Part 5: Handling Nested JSON
7. Practical: CSV to JSON Conversion
8. Real-World Data Engineering Perspective
9. Summary & Quick Revision Sheet
10. General Interview Questions (Beginner / Intermediate / Advanced)
11. Data Engineer Interview Preparation

1. Session Overview & Learning Objectives

In this session, we explored JSON (JavaScript Object Notation) — one of the most widely used data formats in the modern tech world. We covered what JSON is, why it is so popular, how it compares to Python dictionaries, and how to work with it programmatically using Python's built-in **json** module. We also looked at real-world use cases involving student data management and nested JSON structures, and practiced a hands-on CSV-to-JSON conversion pipeline.

Learning Objectives

By the end of this session, students will be able to:

- Explain what JSON is, its structure, and why it is the industry standard for data exchange.
- Differentiate between JSON and Python dictionaries.
- Understand and apply the concepts of Serialization and Deserialization.
- Use the four core JSON functions: **json.dump()**, **json.load()**, **json.dumps()**, and **json.loads()**.
- Navigate and extract data from deeply nested JSON structures.
- Build a practical data pipeline that converts CSV student records into a structured JSON file.
- Explain where JSON fits in real-world data engineering pipelines and APIs.

2. Part 1 — What is JSON?

2.1 Definition

JSON stands for **JavaScript Object Notation**. It is a lightweight, text-based, human-readable format for storing and transporting data. Despite its name, JSON is completely language-independent and is supported by virtually every modern programming language.

■ JSON was derived from JavaScript object syntax, but today it is used by Python, Java, Go, C#, Ruby, Rust, and almost every other language.

2.2 JSON Structure

JSON data is represented as **key-value pairs**, similar to a dictionary in Python. Here is an example of a Swiggy account stored as JSON:

Example: A Swiggy user profile in JSON format

```
{
  "name": "Rohan",
  "email": "rohan.gandhi@gmail.com",
  "city": "Delhi"
}
```

2.3 Why is JSON So Popular?

JSON has become the universal language of data exchange for four key reasons:

Reason	Explanation
1. Human Readable	JSON is plain text. Any developer can read and understand a JSON file without any tools.
2. Language Independent	Python, Java, JavaScript, Go, C#, Rust — every language has built-in or library support for JSON.
3. Lightweight	JSON uses minimal characters compared to older formats like XML, making it faster to transfer.
4. API Standard	REST APIs — the backbone of modern web services — use JSON as the default format for data exchange.

2.4 JSON vs Python Dictionary

JSON and Python dictionaries look very similar but have important differences:

	JSON	Python Dictionary
	Must be strings (always in double quotes)	Can be strings, integers, or tuples
	String, Number, Boolean (true/false), null, Array, Object	Any Python object (including sets, custom objects, etc.)

	true / false (lowercase)	True / False (capitalized)
	null	None
	Not supported	Not officially supported but workarounds exist
	Stored as .json text files	Exists only in memory

■ **Key rule: In JSON, ALL keys must be double-quoted strings. A Python dict can have integers or tuples as keys — JSON cannot.**

Python dict keys that are NOT allowed in JSON

```
# Valid Python Dictionary (NOT valid JSON)

student_info = {

    "name": "Tanishq",    # string key – valid in both

    "age": 25,

    25: "Hey",            # integer key – INVALID in JSON

    (1, 2): "Hey"         # tuple key – INVALID in JSON

}
```

3. Part 2 — Serialization & Deserialization

3.1 Serialization (Packing Data)

Serialization is the process of converting a Python object (e.g., a dictionary, list) into a JSON format so it can be stored in a file or transmitted over a network.

✓ **Think of serialization as packing a suitcase. You take all your items (Python objects) and pack them neatly into a compact, portable format (JSON) that can be transported anywhere.**

Serialization Example

```
# Python dictionary (in memory)

student = {

    "name": "Moksh",

    "marks": 99

}

# Serialization: pack this into JSON format

import json

json_string = json.dumps(student)

print(json_string)    # Output: {"name": "Moksh", "marks": 99}
```

3.2 Deserialization (Unpacking Data)

Deserialization is the reverse process — converting JSON data (from a file or string) back into a Python object such as a dictionary or list.

✓ **Deserialization is like unpacking a suitcase. You receive the JSON data and unpack it back into usable Python objects you can work with in your code.**

Deserialization Example

```
# JSON string received (e.g., from an API or file)

json_string = '{"name": "Abhay Kumar", "marks": 110}'

# Deserialization: unpack JSON back into Python dict

import json
```

```
data = json.loads(json_string)

print(type(data))    # Output:
print(data["name"])  # Output: Abhay Kumar
```

	Functions	Use Case
Object → JSON	json.dump() / json.dumps()	Storing, transmitting
Python Object	json.load() / json.loads()	Reading, processing

4. Part 3 — Working with JSON in Python

Python's built-in `json` module provides four essential functions for working with JSON data. Understanding when to use each function is critical.

	Direction	Primary Use Case
	Python Object → JSON File	Writing to disk
	JSON File → Python Object	Reading from disk
	Python Object → JSON String	Transmitting data (APIs)
	JSON String → Python Object	Receiving/parsing API data

4.1 `json.dump()` — Write Python Object to a JSON File

Use `json.dump()` when you want to save a Python object directly into a file. It takes the Python object and a file object as arguments and writes JSON-formatted data into the file.

`json.dump()` — Writing to a file

```
import json

student_info = {
    "name": "Moksh",
    "marks": 99
}

# Open a file in write mode and dump the data
with open("student.json", "w") as file:
    json.dump(student_info, file)

# After running this, student.json will contain:
# {"name": "Moksh", "marks": 99}
```

- **Line 1:** Import the `json` module.
- **Lines 3-6:** Create a Python dictionary called `student_info`.
- **Lines 8-9:** Open `student.json` in write ('w') mode. `json.dump()` converts the dictionary to JSON and writes it directly into the file.
- The **with** statement ensures the file is automatically closed after writing.

✓ Tip: Use `indent=4` as a third argument for pretty-printing: `json.dump(data, file, indent=4)`

4.2 `json.load()` — Read JSON from a File into Python

Use `json.load()` to read a JSON file and convert its contents back into a Python object (usually a dictionary or list).

`json.load()` — Reading from a file

```
import json

# Open the file in read mode and load the data

with open("student.json", "r") as file:

    data = json.load(file)

print(data)

# Output: {'name': 'Moksh', 'marks': 99}

print(type(data))

# Output:
```

- **Lines 3-4:** Open `student.json` in read ('r') mode and call `json.load()` to parse the file contents.
- The variable **data** now holds a regular Python dictionary.
- You can access keys just like any dictionary: `data['name']` → 'Moksh'

4.3 `json.dumps()` — Convert Python Object to a JSON String

Use `json.dumps()` when you need the JSON as a Python string — for example, to send it in an HTTP API request, log it, or print it.

`json.dumps()` — Convert to JSON string

```
import json

student_info = {

    "name": "Moksh",

    "marks": 99

}

# Convert dictionary to JSON string
```

```
json_string = json.dumps(student_info)

print(json_string)

# Output: {"name": "Moksh", "marks": 99}

print(type(json_string))

# Output:
```

■ **dumps()** = 'dump to string'. The 's' stands for string. It does NOT write to a file.

4.4 json.loads() — Convert JSON String to Python Object

Use **json.loads()** when you receive JSON data as a string (e.g., from an API response or a database field) and need to convert it into a workable Python object.

json.loads() — Parse JSON string into Python dict

```
import json

# A JSON string (as received from an API)
json_string = '{"name": "Abhay Kumar", "marks": 110}'

print(type(json_string))    # Output:

# Deserialize: convert JSON string → Python dict
data = json.loads(json_string)

print(type(data))           # Output:
print(data["name"])         # Output: Abhay Kumar
print(data["marks"])        # Output: 110
```

■ **loads()** = 'load from string'. The 's' stands for string. It reads from a string, NOT a file.

5. Part 4 — Student Data Management

A common real-world use case is managing structured data like student records using JSON. Let's see how to build a complete pipeline — storing and retrieving student information using JSON files.

Student Data Management using JSON

```
import json

# Step 1: Create student records

students = [

    {"id": 1, "name": "Rahul", "maths": 85, "science": 90, "english": 88},

    {"id": 2, "name": "Amit", "maths": 78, "science": 80, "english": 76},

    {"id": 3, "name": "Neha", "maths": 95, "science": 92, "english": 96},

]

# Step 2: Save to a JSON file

with open("students.json", "w") as file:

    json.dump(students, file, indent=4)

# Step 3: Read it back

with open("students.json", "r") as file:

    loaded_students = json.load(file)

# Step 4: Process the data

for student in loaded_students:

    print(f"{student['name']}: Maths = {student['maths']}")
```

Expected Output

```
# Output:

Rahul: Maths = 85

Amit: Maths = 78
```

```
Neha: Maths = 95
```

6. Part 5 — Handling Nested JSON

Real-world JSON data is almost always **nested** — JSON objects inside other JSON objects, or arrays of JSON objects. Understanding how to navigate nested structures is one of the most critical skills for a data engineer.

6.1 What is Nested JSON?

Nested JSON means a JSON value is itself another JSON object or array. For example, an employee record might contain an address object inside it:

Nested JSON — employee with address and skills

```
{  
  "name": "Tanishq",  
  "address": {  
    "city": "Delhi",  
    "state": "Delhi",  
    "pincode": "110001"  
  },  
  "skills": ["Python", "SQL", "Spark"]  
}
```

6.2 Accessing Nested Data

Use chained dictionary access with square brackets to navigate nested JSON:

Accessing Nested JSON Values

```
import json  
  
with open("student.json", "r") as file:  
    data = json.load(file)  
  
# Access a nested key (address → city)  
print(f"{data['name']} lives in city: {data['address']['city']}")  
  
# Output: Tanishq lives in city: Delhi
```

```
# Access an item from an array (employees → second employee)

print(f"Salary of {data['employees'][1]['name']} is {data['employees'][1]['salary']}")

# Output: Salary of Ravi is 75000
```

6.3 Real-World Nested JSON — Cleartrip API Example

APIs like Cleartrip (flight booking) return complex, deeply nested JSON. Here's an example of accessing a deeply nested value:

Real-World API: Navigating Cleartrip Flight Data

```
import json

with open("cleartrip_data.json", "r") as cleartrip_file:

    cleartrip_data = json.load(cleartrip_file)

# Accessing deeply nested data (5 levels deep)

airport_code = cleartrip_data["filterData"]["onwardLeg"]["nearbyAirportFilter"][0]["airportCode"]

print(airport_code)

# Output: BLR (or whichever airport code is in the data)
```

✓ **Pro tip:** When working with deeply nested JSON, use `.get()` to safely access keys and avoid `KeyError` exceptions in production pipelines.

Safe Navigation with `.get()`

```
# Safe access using .get() – avoids KeyError

city = data.get("address", {}).get("city", "Unknown")

print(city) # Output: Delhi (or "Unknown" if key is missing)
```

7. Practical — CSV to JSON Conversion Pipeline

This practical brings together CSV file handling and JSON serialization to build a data transformation pipeline — a fundamental task in data engineering.

7.1 Problem Statement

You are given a CSV file containing student marks. Your task is to build a pipeline that:

- Reads student data from a CSV file.
- Handles missing marks by replacing them with 0.
- Converts all mark values to integers.
- Calculates the total marks and percentage for each student.
- Assigns a grade based on percentage thresholds.
- Saves the final processed data as a JSON file.

7.2 Input Data (CSV)

students.csv — Note: some marks are missing (blank cells)

```
id,name,maths,science,english
1,Rahul,85,90,88
2,Amit,78,,80
3,Neha,95,92,96
4,Priya,,85,89
5,Karan,70,75,
```

7.3 Grading Criteria

	Condition	Remark
	Percentage >= 90%	Excellent
	Percentage >= 75%	Good
	Percentage >= 60%	Average
	Percentage < 60%	Needs Improvement

7.4 Complete Solution

Complete CSV-to-JSON Pipeline

```
import csv
import json
```

```
# Step 1: Read data from CSV

students = []

with open("students.csv", "r") as csv_file:

    reader = csv.DictReader(csv_file)

    for row in reader:

        students.append(row)

# Step 2 & 3: Replace missing marks with 0 and convert to integer

for student in students:

    student["maths"] = int(student["maths"]) if student["maths"] else 0
    student["science"] = int(student["science"]) if student["science"] else 0
    student["english"] = int(student["english"]) if student["english"] else 0

# Step 4: Calculate total marks

    student["total"] = student["maths"] + student["science"] + student["english"]

# Step 5: Calculate percentage

    student["percentage"] = round((student["total"] / 300) * 100, 2)

# Step 6: Determine grade

    p = student["percentage"]

    if p >= 90:

        student["grade"] = "A"

    elif p >= 75:

        student["grade"] = "B"

    elif p >= 60:

        student["grade"] = "C"

    else:
```



```
        student["grade"] = "D"

# Step 7: Write final JSON file

with open("student_results.json", "w") as json_file:

    json.dump(students, json_file, indent=4)

print("Processing complete! student_results.json has been created.")
```

7.5 Expected Output (student_results.json)

student_results.json (partial output)

```
[

  {

    "id": "1",

    "name": "Rahul",

    "maths": 85,

    "science": 90,

    "english": 88,

    "total": 263,

    "percentage": 87.67,

    "grade": "B"

  },

  {

    "id": "2",

    "name": "Amit",

    "maths": 78,

    "science": 0,

    "english": 80,

    "total": 158,

    "percentage": 52.67,
```

```
    "grade": "D"  
  }  
]
```

8. Real-World Data Engineering Perspective

8.1 Why Data Engineers Use JSON

JSON is the lifeblood of modern data engineering pipelines. Here is where you encounter it constantly in a production environment:

- **REST API Responses:** Every API call to services like Stripe, Twilio, Cleartrip, or Google Maps returns JSON.
- **Configuration Files:** Cloud infrastructure tools (Terraform, Kubernetes), CI/CD pipelines, and Spark configurations use JSON.
- **Message Queues:** Kafka topics and RabbitMQ messages are typically serialized as JSON.
- **Data Lake Storage:** Raw landing zones in S3, GCS, or Azure Data Lake store JSON files from event streams.
- **NoSQL Databases:** MongoDB, DynamoDB, Firestore — all store and query JSON-like documents natively.
- **Logging Systems:** Structured logs in Elasticsearch are JSON objects, making them searchable.

8.2 JSON in ETL/ELT Pipelines

Pipeline Stage	JSON Use Case
Extract	Pull JSON from REST APIs, Kafka streams, S3 buckets, webhook payloads
Transform	Parse nested JSON, flatten structures, validate schemas (e.g., using Pydantic or Great Expectations)
Load	Write JSON to databases (MongoDB, BigQuery), convert to Parquet for analytical workloads

8.3 Performance & Scalability Considerations

- **JSON vs Parquet:** For large-scale analytics (100GB+ files), convert JSON to Parquet for 10x faster queries and 5x compression.
- **Schema Evolution:** JSON's flexible schema is a double-edged sword. Use schema registries (Confluent, AWS Glue) in production.
- **Streaming JSON:** Use `ijson` or `orjson` for memory-efficient parsing of large JSON files instead of loading everything into memory.
- **orjson Library:** 2-5x faster than the standard `json` module — preferred in high-throughput production systems.
- **Nested JSON Explosion:** Deeply nested JSON can cause 'schema explosion' in columnar stores. Flatten intelligently before loading to BigQuery or Snowflake.

9. Summary & Quick Revision Sheet

9.1 Core Concepts Summary

Concept	Definition
JSON	JavaScript Object Notation — lightweight text format for data exchange
Key Rule	All JSON keys MUST be strings (double-quoted)
Serialization	Python Object → JSON (packing for storage/transmission)
Deserialization	JSON → Python Object (unpacking for use in code)
json.dump()	Serialize Python object → write to JSON file
json.load()	Read JSON file → deserialize to Python object
json.dumps()	Serialize Python object → JSON string (for APIs)
json.loads()	JSON string → deserialize to Python object
Nested JSON	JSON objects/arrays inside other JSON structures
Safe Access	Use .get('key', default) to avoid KeyError on missing keys

9.2 Syntax Cheat Sheet

JSON Quick Reference — All Four Functions

```
import json

# 1. Write dict to JSON file
with open("data.json", "w") as f:
    json.dump(my_dict, f, indent=4)

# 2. Read JSON file to dict
with open("data.json", "r") as f:
    my_dict = json.load(f)

# 3. Dict → JSON string
json_str = json.dumps(my_dict)

# 4. JSON string → dict
```

```
my_dict = json.loads(json_str)

# 5. Access nested key

value = data["level1"]["level2"]["key"]

# 6. Safe access

value = data.get("level1", {}).get("level2", "default")
```

✓ **Memory trick: dump/load deal with FILES. dumps/loads deal with STRINGS. The extra 's' = String.**

10. General Interview Questions

A. Beginner Level

Q: What does JSON stand for?

A: JSON stands for JavaScript Object Notation. It is a lightweight, text-based, human-readable data interchange format derived from JavaScript object syntax.

Q: What are the valid data types in JSON?

A: JSON supports: String (double-quoted), Number (integer or float), Boolean (true/false — lowercase), null, Array (ordered list), and Object (key-value pairs). Python-specific types like tuples, sets, and None (use null instead) are not valid JSON.

Q: How is JSON different from a Python dictionary?

A: JSON keys must always be strings in double quotes, whereas Python dict keys can be integers, tuples, or strings. JSON uses true/false/null while Python uses True/False/None. JSON is a text format; a dict is an in-memory Python object.

Q: What is the difference between `json.dump()` and `json.dumps()`?

A: `json.dump()` writes a Python object as JSON directly to a file object. `json.dumps()` converts a Python object to a JSON-formatted string. The 's' in dumps stands for 'string'.

Q: What is the difference between `json.load()` and `json.loads()`?

A: `json.load()` reads JSON from an open file object and returns a Python object. `json.loads()` parses a JSON string and returns a Python object. The 's' in loads stands for 'string'.

Q: What is serialization in the context of JSON?

A: Serialization is the process of converting a Python object (like a dictionary or list) into a JSON-formatted string or file so it can be stored, transmitted, or shared across systems.

Q: What is deserialization?

A: Deserialization is the reverse of serialization — it converts a JSON string or file back into a Python object (dictionary, list, etc.) so you can work with the data in Python.

Q: Can you have duplicate keys in a JSON object?

A: Technically the JSON specification does not forbid it, but it is strongly discouraged and leads to undefined behavior. Most parsers will use the last value for a duplicate key.

Q: How do you pretty-print a JSON file in Python?

A: Use the indent parameter: `json.dump(data, file, indent=4)` or `json.dumps(data, indent=4)`. This adds indentation and newlines, making the JSON human-readable.

Q: What error do you get if you try to serialize a Python set to JSON?

A: You get a `TypeError: Object of type set is not JSON serializable`. Sets are not supported by JSON. You must convert the set to a list first: `json.dumps(list(my_set))`.

B. Intermediate Level

Q: How do you handle missing keys safely when parsing nested JSON?

A: Use the `.get()` method with a default value: `data.get('key', 'default')`. For nested access: `data.get('level1', {}).get('level2', None)`. This avoids `KeyError` exceptions.

Q: What is the purpose of the indent parameter in `json.dump()`?

A: The indent parameter controls pretty-printing. `json.dump(data, file, indent=4)` formats the JSON with 4-space indentation, making it human-readable. Without it, the JSON is compact (all on one line).

Q: How do you serialize an object that contains a datetime field?

A: The standard `json` module cannot serialize datetime objects by default. You must either convert datetime to a string (`str(dt)`) before serializing, or use a custom `JSONEncoder` class that overrides the `default()` method.

Q: What is the difference between `json.load()` on a corrupted file vs. a valid file?

A: If the file contains invalid JSON, `json.load()` raises a `json.JSONDecodeError` (a subclass of `ValueError`). Always wrap `json.load()` in a try-except block in production to handle corrupt files gracefully.

Q: How would you flatten a nested JSON structure?

A: You can write a recursive function or use the `pandas json_normalize()` function. Flattening means converting `{'a': {'b': 1}}` into `{'a_b': 1}`, which is needed when loading into tabular systems like BigQuery.

Q: What does `sort_keys=True` do in `json.dumps()`?

A: It sorts the keys of the JSON output alphabetically. This is useful for deterministic output, making it easier to diff JSON files or test expected outputs.

Q: How do you read a JSON file that contains multiple JSON objects (JSON Lines format)?

A: JSON Lines (`.jsonl`) has one JSON object per line. Read the file line by line and call `json.loads()` on each line: `for line in file: data = json.loads(line)`.

Q: What is the difference between a JSON Array and a JSON Object?

A: A JSON Array is an ordered list of values enclosed in square brackets: `[1, 2, 3]`. A JSON Object is an unordered collection of key-value pairs enclosed in curly braces: `{'name': 'Alice'}`. Arrays map to Python lists; objects map to Python dicts.

Q: How would you merge two Python dictionaries and save the result as JSON?

A: Use dict unpacking: `merged = {**dict1, **dict2}` or `merged = dict1 | dict2` (Python 3.9+). Then `json.dump(merged, file, indent=4)`.

Q: What is the `ensure_ascii` parameter in `json.dumps()`?

A: By default, `json.dumps()` escapes all non-ASCII characters as `\uXXXX`. Setting `ensure_ascii=False` allows non-ASCII characters (like Unicode, Hindi, or Chinese characters) to appear directly in the output.

C. Advanced Level

Q: How would you implement a custom JSON encoder for Python dataclasses?

A: Subclass `json.JSONEncoder` and override the `default()` method. Check if the object is an instance of your dataclass, then return `obj.__dict__`. Pass `cls=MyEncoder` to `json.dumps()`: `json.dumps(obj, cls=MyEncoder)`.

Q: What are the performance implications of using the standard json module vs. orjson?

A: The standard json module is implemented in Python (with some C acceleration). orjson is a third-party library written in Rust that is 2-10x faster for both serialization and deserialization. It also natively supports datetime, numpy arrays, and dataclasses. Use orjson in high-throughput pipelines.

Q: Explain the concept of schema validation for JSON data in a production pipeline.

A: In production, you cannot trust incoming JSON to always match the expected structure. Use Pydantic (Python) or JSON Schema (jsonschema library) to validate that required fields exist, data types are correct, and values are within expected ranges before processing.

Q: How does JSON differ from MessagePack and Avro for data serialization in data pipelines?

A: JSON is human-readable but verbose. MessagePack is a binary format that is smaller and faster than JSON but not human-readable. Avro is a binary format with built-in schema evolution — ideal for Kafka streams and Hadoop ecosystems. JSON is best for APIs; Avro/Parquet are better for large-scale data processing.

Q: How would you handle JSON files larger than available RAM?

A: Use streaming JSON parsing with the ijson library, which processes JSON incrementally without loading the entire file into memory. Alternatively, convert the large JSON to NDJSON (newline-delimited JSON) first and process line-by-line.

11. Data Engineer Interview Preparation

■ This section focuses on how JSON is used in REAL production data engineering environments. These questions are asked in Data Engineer interviews at companies like Flipkart, Amazon, Swiggy, Zepto, Meesho, and global tech firms.

Q: How is JSON used in a typical data ingestion pipeline at a tech company?

A: JSON is the most common format for ingesting data from external APIs, webhooks, and event streams. For example, at an e-commerce company, order events are published to a Kafka topic as JSON messages. A consumer service reads these messages, deserializes the JSON, validates the schema, and writes the data to a data warehouse like BigQuery or Snowflake. The pipeline stages are: API/Event Source → JSON Payload → Kafka → Consumer → Validation → Transform → Load.

Q: Why do data engineers prefer Parquet over JSON for large-scale analytics?

A: Parquet is a columnar binary format, while JSON is a row-based text format. For analytics queries that read only a few columns out of 100, Parquet reads only the required columns (columnar access), whereas JSON requires reading the entire record. Parquet also provides 5-10x better compression and 3-5x faster query performance on engines like Spark, Presto, and BigQuery.

Q: How would you handle schema evolution in a JSON-based data pipeline?

A: Schema evolution means the structure of incoming JSON may change over time (new fields added, types changed). Best practices: use a schema registry (Confluent Schema Registry for Kafka), validate incoming JSON against a Pydantic model or JSON Schema and route invalid records to a dead-letter queue, use MERGE statements in the warehouse to handle new columns, and version your schemas.

Q: What is JSON Lines (NDJSON) and when would you use it in a data pipeline?

A: JSON Lines (also called NDJSON or newline-delimited JSON) stores one JSON object per line in a file. This format is ideal for streaming and batch processing because you can process the file line-by-line without loading it entirely into memory. Tools like Apache Spark, dbt, and BigQuery natively support NDJSON. Use it for log files, event streams, and large dataset exports.

Q: How do you flatten deeply nested JSON before loading to a data warehouse?

A: Use `pandas.json_normalize()` for moderate-sized data, or PySpark's `explode()` and `struct` functions for large datasets. For BigQuery, you can also store nested JSON as REPEATED RECORD type and query it with `UNNEST()`. The choice depends on query patterns — flattening is better for analytical queries; nested storage is better for preserving raw event structure.

Q: How would you validate JSON data quality in a pipeline before it reaches the warehouse?

A: Use Great Expectations or Pydantic to define expected schemas and constraints. Check for: required fields presence, correct data types, value ranges, referential integrity, and null handling. Route invalid records to a dead-letter queue (DLQ) in S3 or Kafka for manual review. Log validation failures with metrics to a monitoring system like Datadog or Prometheus.

Q: Explain how JSON is used with REST APIs in a data extraction pipeline.

A: Data engineers build API extractors that send HTTP GET/POST requests to REST APIs (e.g., Stripe for payments, HubSpot for CRM). The API returns a JSON response. The extractor deserializes the JSON, handles pagination (following `next_cursor` or `next_page` tokens in the JSON response), and writes the data to a staging area. Rate limiting, retry logic, and incremental extraction (using `updated_at` timestamps from the JSON) are critical production considerations.

Q: What are common issues when working with JSON from third-party APIs?

A: Common issues include: inconsistent key names (e.g., `userId` vs `user_id`), fields changing type between responses (e.g., `id` coming as string sometimes and integer other times), deeply nested structures that require complex flattening, missing optional fields that cause `KeyError`, large payloads causing memory issues, and API version changes breaking the schema. Always use defensive parsing with `.get()` and validate schemas.

Q: How does DynamoDB handle JSON data, and how is this relevant to data engineers?

A: DynamoDB stores data as JSON-like documents (it uses its own `AttributeValue` format internally). Data engineers often export DynamoDB data for analytical purposes using DynamoDB Streams → Lambda → S3 (as JSON) → Glue Crawler (to infer schema) → Athena or Redshift Spectrum for querying. Understanding how to parse DynamoDB's export format (which wraps values in type descriptors like `{'S': 'hello'}`) is important.

Q: How would you use JSON in an Apache Spark pipeline?

A: Spark can natively read and write JSON: `spark.read.json('s3://bucket/data/*.json')`. Spark infers the schema automatically but you can provide an explicit schema using `StructType` for production reliability. Use `explode()` to flatten JSON arrays into rows, and `get_json_object()` or `from_json()` for parsing JSON strings stored in DataFrame columns. For nested JSON, use `json_tuple()` or select with dot notation on `StructFields`.

Q: What is the role of JSON Schema in data engineering?

A: JSON Schema is a vocabulary for validating the structure and content of JSON data. In data engineering, it is used to: enforce contracts between data producers and consumers, validate API responses before loading, document the expected structure of data, and auto-generate data quality checks. Tools like `jsonschema` (Python), Confluent Schema Registry, and AWS Glue Schema Registry implement JSON Schema validation.

Q: How do you handle null/missing values in JSON during ETL?

A: Strategies include: using `.get()` with defaults during parsing, using `COALESCE` in SQL after loading, replacing null with 0 or 'Unknown' during transformation (as done in the CSV-to-JSON assignment), logging records with critical missing fields to a DLQ, and using `NOT NULL` constraints in the target schema to catch issues early.

Q: Compare JSON with XML for data interchange in enterprise systems.

A: JSON is more compact, faster to parse, and easier for developers to work with. XML supports attributes, comments, namespaces, and document validation via XSD schemas — making it preferred in legacy enterprise systems, government APIs, and SOAP-based web services. For modern REST APIs and data pipelines, JSON is the clear choice. Data engineers sometimes need to parse XML from legacy systems and convert it to JSON for modern pipelines.

Q: What is a dead-letter queue (DLQ) and how does it relate to JSON parsing failures?

A: A dead-letter queue is a storage destination (e.g., an S3 bucket or Kafka topic) where records that fail processing are sent instead of being dropped. In JSON-based pipelines, if a JSON message fails schema validation (e.g., missing required field, type mismatch), it is routed to the DLQ with metadata about the failure. This allows data engineers to investigate issues without losing data and reprocess records after fixing the parsing logic.

Q: How would you build a data pipeline that ingests real-time API data and saves it as structured JSON?

A: Architecture: 1) Trigger: AWS EventBridge cron or Airflow DAG schedules the extraction. 2) Extract: Call the REST API, handle pagination, collect all JSON records. 3) Transform: Deserialize JSON, validate schema, flatten nested fields, handle nulls, calculate derived fields. 4) Load: Write to S3 as NDJSON partitioned by date (e.g., s3://bucket/2024/01/15/data.jsonl). 5) Catalog: Run Glue Crawler to update the schema in the Data Catalog. 6) Query: Use Athena or Spark for downstream analytics. This is the standard Medallion Architecture pattern.